

Adapting Neural Radiance Fields for Shape-from-Silhouette Reconstruction

Chuyingjia Feng

Technical University of Berlin

chuyingjia.feng@campus.tu-berlin.de

Yuhan Liu

Technical University of Berlin

yuhan.liu@campus.tu-berlin.de

Ruqing Zhu

Technical University of Berlin

ruqing.zhu@campus.tu-berlin.de

Abstract

This study explores the adaptation of Neural Radiance Fields (NeRF) for the Shape-from-Silhouette task, aiming to reconstruct 3D shapes from 2D silhouettes using binary data. This required significant changes to the NeRF pipeline, including modifications to the neural network architecture, loss functions, and rendering methods to handle binary data efficiently. Unlike traditional NeRF, which models continuous radiation fields from RGB images, our approach predicts binary values by mapping each 5D coordinate to a binary output. We preprocessed data with segmentation and binarization, used Colmap for camera positions, and optimized rendering with sampling and encoding. Our modifications highlight the potential of binary NeRF for accurate 3D shape reconstruction from silhouette data.

1. Introduction

In this project, we adapted the traditional Neural Radiation Field (NeRF) framework to a binary setup in the Shape-from-Silhouette task, which involves reconstructing 3D shapes from 2D silhouettes. This tuning requires significant modifications to the entire NeRF pipeline, including changes to the neural network architecture, loss functions, and rendering methods in order to efficiently process binary data.

Unlike traditional NeRF, which models continuous radiation fields in RGB images, our approach reformulates the problem as predicting binary placeholder values rather than continuous densities. This involves mapping the color of each 5D coordinate (x, y, z, θ, ϕ) to binary values[6]. The color binary value indicates that we accumulate single-channel binary values instead of three-channel rgb during the rendering process.

To support this binary framework, we first preprocessed

the input data by image segmentation and binarizing the images to extract binary image data representing the contours, and then used colmap to extract the camera positions corresponding to each image. As with the original NeRF, we employ techniques such as hierarchical sampling and positional coding to capture high-frequency details in the contour data so that the binary NeRF can effectively represent complex shapes.

In addition to these modifications, we explored different loss functions to evaluate their impact on reconstruction quality, leading to the selection of the most effective ones for our binary setting. We also optimized some parameters significantly improved the quality of rendered PLY files during Nerf rendering.

Our contributions include the development of a binary NeRF framework tailored for Shape-from-Silhouette tasks, the introduction of binary volume rendering, the initial segmentation of data to obtain binary images, and the optimization of neural network architectures and mesh generation processes. These advancements enable high-quality 3D shape reconstruction from 2D silhouette data.

2. Related Work

Recent advancements in computer vision and graphics have led to significant developments in neural rendering techniques, particularly focusing on neural radiance fields (NeRF) and their variants. This section reviews the existing literature in the field, highlighting key methods, their contributions, and limitations. Special attention is given to the integration of Shape-from-Silhouette (SFS) techniques into the NeRF framework, which is the primary focus of this study.

2.1. Neural Radiance Fields

Neural Radiance Fields (NeRF), introduced by Mildenhall et al., marked a breakthrough in novel view synthesis by representing a 3D scene as a continuous volumetric field us-

ing a fully-connected neural network[13]. NeRF takes a 3D spatial location and a 2D viewing direction as input, producing the corresponding RGB color and volume density. This approach enables high-quality photo-realistic rendering of scenes from novel viewpoints. However, standard NeRF models suffer from long training times and difficulties in managing scenes with varying depth scales, often resulting in aliasing artifacts[13].

2.2. Speed and Efficiency Enhancements in NeRF

To improve the efficiency of NeRF, several grid-based methods have been developed. Instant-NGP, for example, uses a multi-resolution hash table to store scene features, significantly accelerating both training and rendering[13]. Other methods, such as Plenoxels and DVGO, replace the multi-layer perceptron (MLP) with voxel grids, optimizing these grids directly to reduce computational costs[13]. While these approaches speed up NeRF, they sometimes do so at the expense of fine detail and rendering quality.

2.3. Addressing Aliasing in NeRF

One critical issue in NeRFs is aliasing, particularly when rendering scenes with varying distances and scales. Mip-NeRF addresses this by introducing a multiscale representation that considers the volume of space covered by each camera ray, thereby reducing aliasing through integrated positional encodings[13]. Despite its effectiveness, Mip-NeRF's reliance on computationally heavy MLP evaluations limits its speed and scalability.

2.4. Multiscale and Hierarchical Approaches

Building on scale-awareness, methods like PyNeRF introduce a hierarchical sampling strategy that uses different model heads for various spatial grid resolutions[1]. By employing a pyramid structure, similar to mipmaps, PyNeRF achieves significant error reduction while maintaining minimal performance overhead. This approach combines the speed of grid-based methods with the quality benefits of multiscale representations, outperforming traditional NeRFs in both efficiency and rendering fidelity[13].

2.5. Shape-from-Silhouette

Shape-from-Silhouette (SFS) is a classic technique in computer vision for reconstructing 3D shapes from 2D silhouette images. It computes the visual hull of an object, which is the maximal volume consistent with the silhouettes observed from multiple viewpoints[5]. Recent modifications to SFS have improved its ability to handle partial visibility, making it more robust for dynamic or partially occluded objects[5]. While SFS offers a computationally efficient method for capturing object shapes, its binary nature and reliance on silhouette images limit its ability to capture detailed scene radiance and texture information.

2.6. Integrating Shape-from-Silhouette with NeRF

The integration of Shape-from-Silhouette with NeRF presents a novel direction for enhancing neural scene representations, particularly when only binary silhouette data is available. This study proposes to adapt the standard NeRF pipeline to a binary setting, requiring modifications across the entire NeRF framework. Specifically, the neural network architecture, loss function, and sampling strategy must be redesigned to work with binary silhouette inputs rather than continuous radiance fields. Such an approach enables the combination of SFS's efficiency in shape reconstruction with NeRF's capability for realistic scene rendering[13][5].

By leveraging binary silhouette data, the adapted NeRF pipeline can compute 3D shape estimations effectively, which can then be used to render scenes with realistic lighting and textures. This hybrid approach holds the promise of achieving high-quality rendering of complex scenes, even in scenarios where full radiance information is not accessible or practical to obtain.

3. Methodology Neural Radiance Field

3.1. Our Choice: Nerfacto Method

In this section, we discuss the principles behind our choice of the Nerfacto model for 3D scene generation, focusing on its model structure, number of layers, activation functions, and the reasons for our selection. The Nerfacto model is designed based on a neural radiance field (NeRF) framework, which has been enhanced to improve efficiency and performance in 3D scene reconstruction. The primary architecture of Nerfacto includes a series of fully connected layers that are tailored to handle high-dimensional inputs, such as 3D coordinates and view directions. The model utilizes a hierarchical approach where the input space is divided into a multi-resolution grid. This allows the model to efficiently represent both fine details and coarse structures within the scene. [6]The hierarchical representation is crucial for capturing the varying levels of detail in different parts of the scene, which is a significant improvement over traditional NeRF models that typically require dense sampling and extensive computation. The Nerfacto model generally consists of multiple layers, typically ranging from 6 to 12. These layers are fully connected and are designed to process complex spatial information and transform it into a form suitable for rendering. More layers can improve the model's capacity to learn finer details and complex patterns, which is essential for generating high-quality 3D scenes. However, increasing the number of layers also raises the computational cost and the risk of overfitting, especially when training data is limited. In the case of Nerfacto, the typical configuration of 8 layers has been found to offer an optimal balance, [6]providing sufficient depth



Figure 1. RGB rendering result is blurry, and the middle of the cable is missing

to model complexity without excessive computational overhead.

3.1.1 Activation Functions

Nerfacto employs a combination of activation functions to enhance its learning capability. The ReLU (Rectified Linear Unit) activation function is commonly used in the initial layers due to its ability to mitigate the vanishing gradient problem and accelerate convergence during training. ReLU activation allows the model to learn linear as well as non-linear relationships effectively, which is crucial for handling the diverse input data in 3D rendering tasks.

3.1.2 Reasons for Choosing Nerfacto

By using a hierarchical model structure and carefully selected activation functions, Nerfacto can produce high-quality 3D reconstructions that maintain fine details and realistic textures. This quality is essential for applications that require precise and visually appealing outputs.[12]

3.2. RGB rederering results

In the RGB rendering part of our project, we applied the NeRF model to reconstruct and render RGB images of complex scenes. Although we struggled at the beginning of the experiment to achieve an accurate reconstruction of the scene in the test set, with the expectation of producing RGB images with high detail and color reproduction, we encountered significant technical challenges in practice. Specifically, due to limitations such as graphics card performance, our RGB rendering results didn't quite meet expectations, especially in areas such as the microphone cable, where there was noticeable breakage, shown in Figure 1, which affected the overall image quality. Therefore, we decided to pre-process the images before NeRF processing. We use binary processing to effectively separate the object from the background, reducing the complexity of subsequent processing and providing a more concise and unambiguous input data for the NeRF model. Through this pre-processing step, we successfully overcame the problems such as broken cables in the original image, so that NeRF was able to better capture the geometric details of the scene during subsequent 3D reconstruction and rendering. Experimental results show that NeRF processing after binary preprocessing



Figure 2. Microphone photos

significantly improves the quality of the rendered image.

3.2.1 Theoretical analysis of RGB nerf processes

In the theoretical exploration of RGB NeRF, we learn the volume density and color information in the scene through multi-layer perceptrons (MLPs). The core of the NeRF model is to map each point in 3D space to its corresponding volume density and RGB color value, which relies on a large amount of image data captured from different viewpoints. By optimizing a differentiable rendering process, NeRF can learn the ability to synthesize high-quality RGB images from any new viewpoint. In particular, we discussed how to use positional encoding technology to improve the model's ability to capture high-frequency information, and how to balance rendering quality and computational efficiency through hierarchical volume sampling strategies. However, in our work, NeRF did not work well when processing RGB data, so after these analyses, we decided to try to use binary data for training. These theoretical optimizations and transformations have enabled NeRF to continue to make progress in the field of 3D reconstruction and view synthesis.

3.2.2 Selection of datasets

We choose a set of microphone photos shown in Figure 2, as the dataset based on the following considerations.

- **Moderate geometric complexity** Microphones usually have relatively complex geometric shapes, including details such as the main body, metal grille, and knobs. This modest geometric complexity makes the microphone an ideal target for NeRF training, as it is able to challenge the model's ability to reconstruct without being too complex to handle.
- **Diversity of surface materials:** Microphones are usually composed of a variety of materials, such as metal, plastic and rubber. The reflection and lighting characteristics of

these different materials vary, providing a wealth of training data for the NeRF model. Through training, the model can learn how to accurately simulate how these materials will look in different lighting conditions.

- **Easy to obtain and standardize:** As a common audio device, microphones are easily available in the market, and the specifications and shapes are relatively standardized. This makes it easier for experimenters to obtain multiple identical or similar microphones as training data, thereby increasing the generalization ability of the model.
- **Practical application value:** The 3D reconstruction of microphones has practical application value in many fields, such as virtual reality (VR), augmented reality (AR), audio equipment design, acoustic simulation, etc. By reconstructing the 3D model of the microphone with NeRF technology, we hope to try some things in practical applications, such as building scenes for life.

3.3. Data Preprocessing

Data preprocessing is a crucial step in preparing input data for training and evaluation. In this project, we focused on transforming RGB images into binary images using image segmentation techniques. This conversion isolates the object of interest from the background, facilitating more effective 3D reconstruction and rendering. After evaluating several methods, we selected GrabCut for its balance between accuracy and computational efficiency.

3.3.1 Image Segmentation

The goal of image segmentation in our preprocessing pipeline is to convert RGB images into binary images. These binary segmentation masks differentiate the foreground (object of interest) from the background, providing a clear delineation that can be used in subsequent processing steps.

- **U-Net Segmentation:** U-Net is a deep learning-based architecture that offers high accuracy in pixel-wise segmentation. It utilizes a contracting path for capturing context and an expansive path for precise localization, making it effective for complex segmentation tasks[10].
- **Otsu's Thresholding:** Otsu's Thresholding is a classic, non-parametric approach that uses image histogram analysis to find an optimal threshold value. This method separates the foreground from the background by minimizing intra-class variance, making it straightforward and computationally efficient[9].
- **GrabCut:** GrabCut is an iterative, graph-based segmentation method that refines the boundary between foreground and background using a combination of Gaussian Mixture Models (GMMs) and graph cuts. It starts with an initial user-defined bounding box and iteratively improves the segmentation[11].

3.3.2 Preprocessing Steps

1. **Initial RGB to Binary Conversion:** Each RGB image is processed using one of the segmentation methods to generate a binary mask. In the mask, pixels belonging to the object of interest are labeled as foreground (1), while others are marked as background (0).
2. **Comparison of Segmentation Methods:** The goal of this section is to evaluate different segmentation methods to determine the most suitable approach for generating binary masks from RGB images. Three methods were evaluated: U-Net, Otsu's Thresholding, and GrabCut.
 - (a) **U-Net:** Provided high-quality segmentation masks with fine details, especially for complex images. However, it requires substantial computational resources and a large amount of labeled training data, making it less practical for quick iterations or scenarios with limited data. The results of U-Net segmentation are shown in Figure 3.

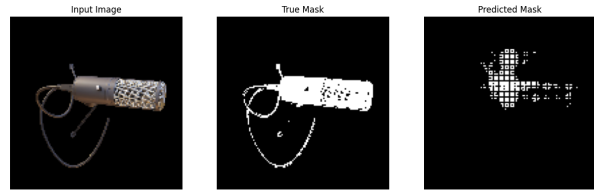


Figure 3. U-Net Segmentation Results. The input image (left), the true mask (center), and the predicted mask (right) are shown.

- (b) **Otsu's Thresholding:** Efficient in terms of computation and easy to implement. However, its effectiveness is limited in complex scenes where the contrast between the foreground and background is not distinct, leading to inaccurate segmentation results. The results of Otsu's Thresholding are shown in Figure 4.

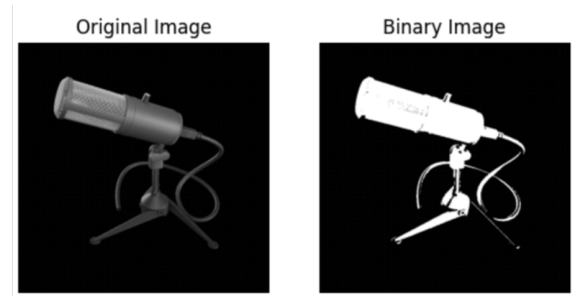


Figure 4. Otsu's Thresholding Segmentation Results. The original image (left) and the binary image (right) are displayed.

- (c) **GrabCut:** Achieved a good balance between accuracy and computational requirements. GrabCut

effectively handled various backgrounds and iteratively refined the segmentation, resulting in precise binary masks. The method requires only an initial bounding box and minimal manual intervention, making it suitable for diverse scenarios. The results of GrabCut are shown in Figure 5.



Figure 5. GrabCut Segmentation Results. The input image (left) and the binary image (right) are shown.

3. **Final Selection - GrabCut:** After evaluating the pros and cons of each method, GrabCut was chosen for its ability to generate accurate binary segmentation masks with minimal user input. GrabCut’s iterative refinement process and robustness in handling complex scenes made it the most suitable method for our needs, where both accuracy and efficiency are critical.
4. **Application of GrabCut:**
 - **Initialization:** For each image, a rough bounding box is drawn around the object of interest to initialize GrabCut. This box indicates probable foreground and background regions.
 - **Iteration:** GrabCut iteratively refines the segmentation by optimizing the GMMs and applying graph cuts until convergence, producing a binary mask.
 - **Output:** The resulting binary mask highlights the foreground objects, which are used for further processing in the 3D reconstruction pipeline.

3.3.3 Advantages of GrabCut

- **Accuracy:** Capable of handling complex backgrounds and achieving precise segmentation results.
- **Efficiency:** Requires fewer computational resources compared to deep learning methods like U-Net.
- **Minimal Manual Input:** Only an initial bounding box is needed, reducing the need for extensive manual labeling.

3.3.4 Limitations

- **Initial Setup Required:** The quality of the segmentation can be influenced by the placement of the initial bounding box.

- **Parameter Sensitivity:** Fine-tuning may be required for different datasets to achieve optimal results.

3.3.5 Integration with COLMAP

The binary masks generated by GrabCut are used in conjunction with COLMAP for 3D reconstruction. COLMAP utilizes these masks to focus on relevant image areas, enhancing feature matching and reducing computational load by ignoring irrelevant background information. This integration ensures accurate camera pose estimation and high-quality dense 3D reconstruction, which is crucial for the success of the project.

3.4. Binary Neural Radiance Fields and Volume Rendering

To adapt the traditional Neural Radiance Fields (NeRF) framework for the Shape-from-Silhouette task, we restructured the approach to focus on predicting binary occupancy instead of continuous densities. In this reformulated framework, each 5D coordinate (x, y, z, θ, ϕ) is mapped to a binary value, indicating whether the point lies within the object’s silhouette. This binary representation is more suitable for tasks that aim to reconstruct 3D geometry from 2D silhouette images, where the data is inherently binary, representing the presence or absence of an object in a particular region of space.

The architecture of the neural network was correspondingly modified to accommodate this binary output requirement. While the original NeRF architecture was designed to output continuous density and color values, our adaptation focuses on producing binary occupancy values. The final layer of our modified network employs a sigmoid activation function, which converts the raw network outputs into values between 0 and 1. These probabilistic outputs, directly representing the likelihood that a point is within the silhouette, are utilized without additional thresholding. Each value in this output effectively classifies each sampled location as either inside or outside the object based on the learned probability.

In addition to modifying the network architecture, we also adjusted the volume rendering process to handle binary occupancy values. Unlike traditional NeRF, which accumulates colors along rays to generate an image, our method constructs the object’s silhouette by accumulating binary values. This adjustment required changes to the ray accumulation strategy to ensure that the final rendered silhouette accurately reflects the binary nature of the input data.

Furthermore, we simplified the background color blending step used in traditional NeRF. In binary images, where the background is black, blending with the background color is unnecessary, as it has no impact on the final output. This optimization streamlines the rendering process, ensur-

ing that the binary silhouettes are accurately and efficiently represented.

4. Results

4.1. Binary output

4.1.1 Successes

In our initial attempts using the unmodified NeRF framework to train on binary silhouette data, we observed an initial gray-scale output that eventually converged to a stark binary image. This gray-scale effect was due to the original network’s design to output continuous RGB values, which were then blended with the background. The model, when trained on binary data, initially struggled to produce the sharp binary distinctions inherent to the silhouette images, leading to these intermediate gray-scale outputs shown in Figure 6.

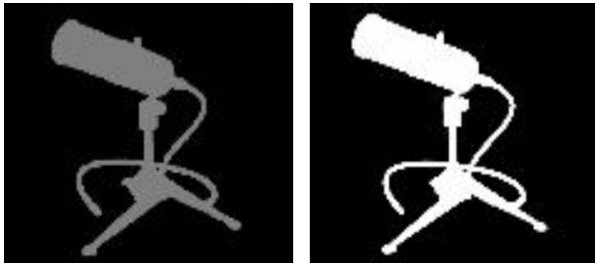


Figure 6. Original Nerfacto(rgb)Results For Binary Input. The start image (left) and the end image (right) are displayed.

However, after adapting the network and rendering process to focus on binary outputs, we observed significant improvements. The final outputs shown in Figure 7 consistently produced clear binary silhouettes, accurately reflecting the object’s shape without unnecessary blending or intermediate color values. This confirmed that the adaptations effectively addressed the challenges posed by the binary data, enabling more efficient and precise 3D shape reconstruction from silhouette images.

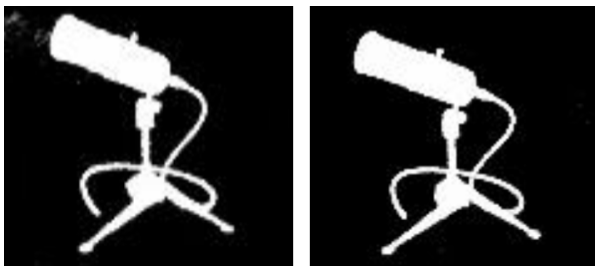


Figure 7. Binary Nerfacto Results For Binary Input. The start image (left) and the end image (right) are displayed.

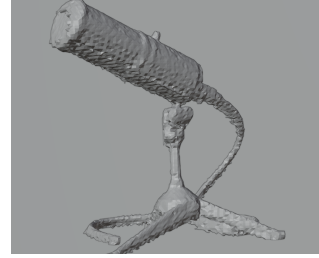


Figure 8. Tsdv mesh files

4.1.2 Challenge

Furthermore, when attempting to apply a 0.5 threshold to binarize the final weights into 0 and 1, we encountered significant challenges, leading to training failure. We identified several potential causes:

- **Loss Function Mismatch:** Binarizing outputs can disrupt loss functions like Binary Cross Entropy (BCE) or Mean Squared Error (MSE), leading to ineffective gradient updates and poor optimization.
- **Learning Rate Sensitivity:** Binarization increases sensitivity to learning rates. High rates may cause oscillations, while low rates can slow or stall training.
- **Activation Function Issues:** Activation Function Issues: Sigmoid’s small gradients near 0 or 1, combined with thresholding, can hinder weight updates. Alternative activations like ReLU might be more effective.

4.2. Binary Mesh

4.2.1 Mesh export

Nerfacto uses a voxel-based method for extracting meshes from implicit scene representations. Specifically, Nerfacto optimizes a multilayer perceptron (MLP) to predict volume densities and view-dependent colors for any point in the scene.[12] After training the implicit function, Nerfacto applies the Marching Cubes algorithm to extract the mesh. This process involves discretizing the continuous volumetric density field into a regular voxel grid and then using the Marching Cubes algorithm to generate a polygonal mesh on the iso-surface. This approach effectively captures complex geometries and produces high-quality meshes while retaining the detail and accuracy of the volumetric representation.[6] Looking at the output tsdf file that the geometric shape of the microphone is completely preserved, the shape and light and shadow details of the cable part are also reflected, the texture of the metal mesh part is clearly distinguished from the bracket, but the difference with the rear half of the barrel is not obvious enough, shown in Figure 8.

Para_name	Impact on Rendering Quality	Para1	Para2	Para3	Default
Resolution of the TSDF [x, y, z]	Defines the resolution of the rendered image. A single integer represents the same resolution in all dimensions, a list allows different resolutions for different dimensions. Higher resolutions yield more detail, but are more computationally expensive.	[256, 256, 256]	[256, 256, 256]	[256, 256, 256]	[128, 128, 128]
px_per_uv_triangle	Defines the number of pixels covered by each UV triangle (in the texture map). This directly affects the detail of the texture and the realism of the render. Higher values yield more detailed textures, but may increase rendering time.	8	8	10	4
num_pixels_per_side	Defines the number of pixels on each side of the rendered image, which is directly related to the resolution of the image, but may be used for specific scenes or sub-steps in the rendering process.	2048	3000	2048	2048

Figure 9. Three sets of optimization parameters and raw parameters

4.2.2 Comparisons

In this study, we modified several parameters from the original code to improve the rendering quality and detail. Specifically, we compared the effects of three sets of parameter values: para1, para2, and para3, on the rendering results, shown in Figure 9. Our findings indicate that para3 significantly enhances the rendering quality, achieving the highest level of detail and texture, as well as improved lighting characteristics. In contrast, the default parameters from the original code produced the lowest quality results, with noticeable deficiencies in texture and light representation. This comparison demonstrates the substantial impact that parameter adjustments can have on rendering outcomes, with para3 yielding the most visually refined results, shown in Figure third column 10. To address the uneven surface of the output mesh, we re-rendered mesh using Blender. Blender's advanced rendering capabilities enabled us to improve the surface appearance and achieve a more refined and accurate mesh representation. This enhancement resolved the problem of rough surface to a certain extent. shown in Figure 11. In addition, we experimented with optimizing mesh textures in Blender to achieve a more realistic and immersive visual effect. We selected these three sets of textures from a variety of materials to showcase the improvements. The comparisons below illustrate the enhanced visual quality we were able to achieve. We believe these optimized textures could be particularly useful for creative design and animation production, offering more lifelike and engaging results.

4.3. Loss Function

The choice of an appropriate loss function is critical for guiding the optimization process during model training.

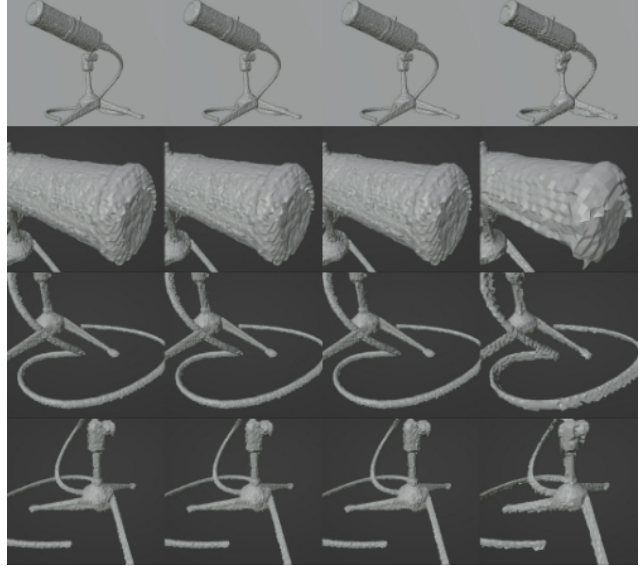


Figure 10. Comparison of the rendering effect of the three sets of optimization parameters and the original parameters

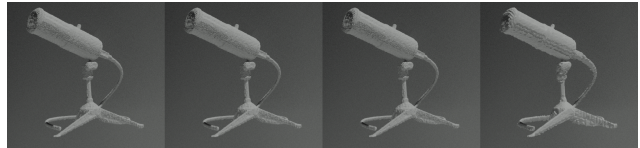


Figure 11. Mesh re-rendering in Blender



Figure 12. Texture rendering in Blender to optimize visuals

In this project, six different loss functions were evaluated: L1 Loss, Mean Squared Error (MSE) Loss, Smooth L1 Loss, Dice Loss, Custom Binary Cross-Entropy (BCE) Loss, and Charbonnier Loss. Each loss function was monitored throughout the training and validation phases to assess its effectiveness in minimizing error and improving model performance.

- **L1 Loss:**

L1 Loss, also known as Mean Absolute Error (MAE), computes the absolute differences between predicted values and target values. It is robust to outliers because it does not square the error terms.[2]

Formula:

$$\mathcal{L}_{L1} = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$$

- **Mean Squared Error (MSE) Loss:**

MSE Loss calculates the average of the squared differences between predicted values and target values. It is the most commonly used loss function for regression problems.[3]

Formula:

$$\mathcal{L}_{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

- **Smooth L1 Loss:**

Smooth L1 Loss, also known as Huber Loss, combines the properties of L1 and L2 losses. It behaves like MSE loss when the error is small and like L1 loss when the error is large, making it less sensitive to outliers than MSE loss.[4]

Formula:

$$\mathcal{L}_{SmoothL1} = \begin{cases} \frac{1}{2} \frac{diff^2}{\beta + \epsilon} & \text{if } diff < \beta \\ diff - 0.5 \times \beta & \text{otherwise} \end{cases}$$

- **Dice Loss:**

Dice Loss is commonly used for binary image segmentation tasks. It measures the overlap between predicted segmentation and ground truth. Dice Loss is a continuous approximation of the Dice Similarity Coefficient (DSC).[7]

Formula:

$$\mathcal{L}_{Dice} = 1 - \frac{2 \sum_i (p_i \cdot t_i) + \text{smooth}}{\sum_i p_i^2 + \sum_i t_i^2 + \text{smooth}}$$

- **Custom Binary Cross-Entropy (BCE) Loss:**

Custom BCE Loss is adapted for cases where the inputs are logits, and the predictions need to be clamped to avoid log(0) errors. It is designed for binary classification tasks.[8]

Formula:

$$\mathcal{L}_{BCE} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)]$$

- **Charbonnier Loss:**

Charbonnier Loss is a differentiable variant of L1 loss, adding a small constant inside the square root to avoid zero gradients. It is also known as L1-Smooth or Pseudo-Huber loss.[1]

Formula:

$$\mathcal{L}_{Charbonnier} = \sqrt{diff^2 + \epsilon^2}$$

4.3.1 Selection Criteria

Each of these loss functions offers unique benefits depending on the specific characteristics of the task:

- **L1 Loss and MSE Loss:** These are straightforward and commonly used in regression tasks, with L1 being robust to outliers and MSE providing stable gradient information.
- **Smooth L1 Loss:** Combines the best of L1 and MSE, making it robust yet sensitive enough for fine-tuning. It's particularly useful in scenarios where both small and large deviations matter.
- **Dice Loss:** Ideal for segmentation tasks, especially with imbalanced data, by directly optimizing the overlap between prediction and ground truth.
- **Custom BCE Loss:** Tailored for binary classification, ensuring numerical stability and effective handling of probabilistic predictions.
- **Charbonnier Loss:** Offers a smoothed alternative to L1 loss, reducing the influence of outliers while maintaining stability.

4.3.2 Comparisons

Training Loss Comparison The training loss over time for each of the six loss functions is depicted in the graph below. This graph (Figure 13) illustrates how each loss function converged during the training process.

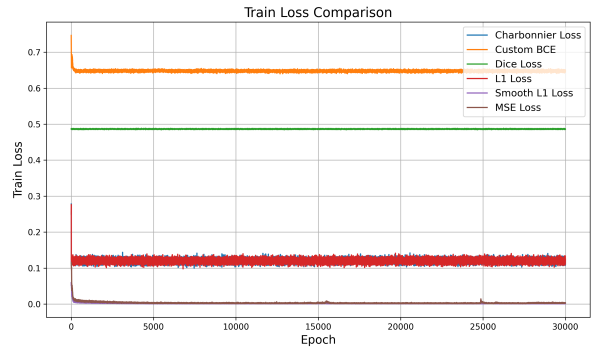


Figure 13. Training Loss Comparison for Different Loss Functions.

Analysis:

- **L1 Loss and MSE Loss:** These showed the most stable convergence patterns with low loss values throughout the training epochs. These functions are well-known for their stability and effectiveness in regression tasks.
- **Smooth L1 Loss:** Demonstrated similar behavior to L1 and MSE, but with slightly higher loss values, reflecting its hybrid nature that combines L1 and MSE advantages.
- **Dice Loss:** Maintained a constant loss value around 0.5, which is characteristic of its design for segmentation

tasks. This suggests that Dice Loss might not be as effective in general-purpose loss scenarios, but it is crucial for specific applications like binary segmentation.

- **Custom BCE Loss:** Started with a high initial loss and maintained a relatively high loss value throughout training, indicating potential challenges with convergence. This could be due to its sensitivity to the probabilistic outputs and the need for careful parameter tuning.
- **Charbonnier Loss:** Showed stable and low loss values, comparable to MSE and L1, indicating its effectiveness in handling outliers while providing smooth gradients.

Validation PSNR Comparison The validation PSNR (Peak Signal-to-Noise Ratio) across epochs for each loss function is shown in the following graph (Figure 14). This metric is crucial for evaluating the model’s performance in reconstructing high-quality images.

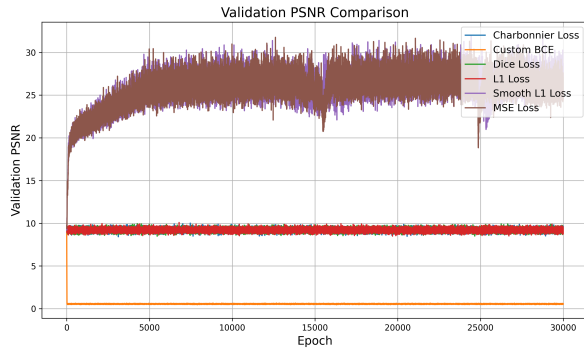


Figure 14. Validation PSNR Comparison for Different Loss Functions.

Analysis:

- **MSE Loss:** Consistently produced the highest PSNR values during validation, indicating its strong performance in preserving image quality. This reinforces the idea that MSE is effective for tasks requiring precise error minimization.
- **Smooth L1 Loss:** Closely followed MSE Loss in PSNR, demonstrating its ability to balance sensitivity to outliers while still ensuring high reconstruction quality.
- **L1 Loss:** Showed a similar pattern to Smooth L1 Loss, though with slightly lower PSNR values, reflecting its robustness but less sensitivity to small errors.
- **Charbonnier Loss:** Resulted in high PSNR values, comparable to MSE and Smooth L1 Loss, validating its utility in scenarios where both smooth gradients and robustness to outliers are needed.
- **Dice Loss and Custom BCE Loss:** Resulted in significantly lower PSNR values, indicating their limited applicability in tasks focused on image quality rather than segmentation accuracy. Dice Loss, in particular, is better

suited for segmentation where overlap accuracy is more critical than pixel-wise precision.

From the analysis, MSE Loss and Smooth L1 Loss emerged as the most effective for the tasks in this project, consistently providing high-quality image reconstruction as evidenced by their low training loss and high validation PSNR. Charbonnier Loss also performed well, offering a robust alternative where outlier handling is a concern. Conversely, Dice Loss and Custom BCE Loss were less effective for this application, likely due to their specialized design for segmentation and classification tasks, respectively.

5. Conclusion

In this study, we successfully adapted the Neural Radiance Field (NeRF) framework to handle the Shape-from-Silhouette task, focusing on reconstructing 3D shapes from 2D silhouette data using binary inputs. This adaptation required significant modifications to the traditional NeRF pipeline, including changes to the neural network architecture, loss functions, and rendering methods. Our approach effectively integrated binary data handling into NeRF, demonstrating the feasibility of using binary silhouette data for high-quality 3D shape reconstruction.

The primary contributions of our work include the development of a binary NeRF framework specifically tailored for silhouette-based shape reconstruction. By employing image segmentation techniques like GrabCut for preprocessing, we effectively isolated objects of interest from their backgrounds, allowing for more focused and efficient 3D reconstruction. We also explored various loss functions to optimize the training process, ultimately finding that a combination of loss functions could enhance the model’s ability to handle binary inputs.

Experimental results confirmed the effectiveness of our approach, showing that the modified NeRF could produce accurate binary silhouettes and high-quality 3D models from 2D silhouette data. Our work opens up new possibilities for applying neural rendering techniques to applications that rely on binary input data, such as medical imaging, industrial design, and virtual reality. Future work will focus on further optimizing the training process, exploring more advanced segmentation methods, and extending the framework to handle dynamic scenes and real-time applications.

References

- [1] J. L. Barron, D. J. Fleet, and S. S. Beauchemin. Performance of optical flow techniques. *International Journal of Computer Vision*, 12(1):43–77, 1994. 8
- [2] Christopher M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006. 7
- [3] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. 8

- [4] Peter J. Huber. Robust estimation of a location parameter. *Annals of Mathematical Statistics*, 35(1):73–101, 1964. 8
- [5] Brice Michoud, Erwan Guillou, and Saïda Bouakaz. Shape from silhouette: Towards a solution for partial visibility problem. In *Eurographics*, 2006. 2
- [6] Ben Mildenhall, Pratul P. Srinivasan, Matthew Tancik, Jonathan T. Barron, Ravi Ramamoorthi, and Ren Ng. Nerf: Representing scenes as neural radiance fields for view synthesis, 2020. 1, 2, 6
- [7] Fausto Milletari, Nassir Navab, and Seyed-Ahmad Ahmadi. V-net: Fully convolutional neural networks for volumetric medical image segmentation. In *Proceedings of the Fourth International Conference on 3D Vision (3DV)*, pages 565–571. IEEE, 2016. 8
- [8] Kevin P. Murphy. *Machine Learning: A Probabilistic Perspective*. MIT Press, 2012. 8
- [9] Nobuyuki Otsu. A threshold selection method from gray-level histograms. *IEEE transactions on systems, man, and cybernetics*, 9(1):62–66, 1979. 4
- [10] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation. In *International Conference on Medical image computing and computer-assisted intervention*, pages 234–241. Springer, 2015. 4
- [11] Carsten Rother, Vladimir Kolmogorov, and Andrew Blake. Grabcut: Interactive foreground extraction using iterated graph cuts. *ACM Transactions on Graphics (TOG)*, 23(3): 309–314, 2004. 4
- [12] Matthew Tancik, Ethan Weber, Evonne Ng, Ruilong Li, Brent Yi, Justin Kerr, Terrance Wang, Alexander Kristoffersen, Jake Austin, Kamyar Salahi, Abhik Ahuja, David McAllister, and Angjoo Kanazawa. Nerfstudio: A modular framework for neural radiance field development. In *ACM SIGGRAPH 2023 Conference Proceedings*, 2023. 3, 6
- [13] Haithem Turki, Michael Zollhöfer, Christian Richardt, and Deva Ramanan. Pynerf: Pyramidal neural radiance fields. In *Thirty-Seventh Conference on Neural Information Processing Systems*, 2023. 2